Agentic架构7: 黑板系统(Blackboard Systems)

> **黑板系统(Blackboard Systems)** —— 一种用于**协调多个专业智能体**的强大且高度灵活的模式。该架构的灵感来源于一组人类专家围绕一块物理黑板协作解决复杂问题的场景。与采用严格预定义的智能体交接顺序不同，黑板系统具有一个**中央共享的数据存储区**（即“黑板”），所有智能体都可以从中**读取问题的当前状态**，并贡献自己的输出。一个动态的**控制器**负责观察黑板，并根据**推进解决方案所需的内容**，决定下一步**激活哪个专业智能体**。这使得工作流呈现出**机会主义和涌现式**的特点。

定义

黑板系统 是一种多智能体架构，其中**多个专业智能体通过一个名为“黑板”的共享中央数据存储库进行读写操作来实现协作**。**控制器或调度器**根据黑板上解决方案的演化状态，**动态决定**下一步应由哪个智能体执行操作。

高级工作流程

1. **共享内存（黑板）：** 一个**中央数据结构保存问题的当前状态**，包括用户请求、中间发现以及部分解决方案。
2. **专业智能体：** 一组**独立的智能体**，每个智能体都有**特定的专长**，持续监控黑板。
3. **控制器：** 一个**中央“控制器”智能体**监控黑板。其职责是分析当前状态，并决定由哪个专业智能体最有可能做出下一步贡献。
4. **机会激活：** **控制器激活被选中的智能体**。该智能体从黑板读取相关数据，执行其任务，并将其发现写回黑板。
5. **迭代：** 此过程重复进行，控制器以动态顺序激活不同的智能体，直到其判定黑板上的解决方案已经完整。

适用场景 / 应用

- **复杂、结构不良的问题：** 适用于**解决路径无法预先确定**、需要**涌现式或机会主义**策略的问题（例如复杂诊断、科学发现）。
- **多模态系统：** **协调处理不同类型数据（文本、图像、代码）的智能体**的绝佳方式，因为它们都可以将各自的发现发布到共享黑板上。
- **动态意义构建：** 需要综合来自多个**不同、异步来源**的信息的场景。

优势与劣势

- **优势：**
  - **灵活性与适应性：** 工作流并非硬编码，而是**根据问题动态涌现**，使系统具有高度适应性。
  - **模块化：** 无需重新架构整个系统，即可轻松添加或移除专业智能体。
- **劣势：**
  - **控制器复杂性：** 整个系统的智能程度在很大程度上取决于**控制器的精妙程度**。一个设计不佳的控制器可能导致低效或循环行为。
  - **调试挑战：** 工作流的非线性、涌现性特征有时使得其**追踪和调试**比简单的顺序流程更加困难。

阶段 0：基础与环境搭建

从标准的搭建流程开始：安装库，并为 Nebius、LangSmith 和 Tavily 配置 API 密钥。

步骤 0.1：安装核心库

安装本系列项目所需的标准库套件。

```
pip install -q -U langchain-nebius langchain langgraph rich python-dotenv
langchain-tavily
```

步骤 0.2：导入库并配置密钥

导入必要的模块，并从 `.env` 文件中加载 API 密钥。

需要操作： 请在此目录下创建一个 `.env` 文件，并填入密钥：

```
NEBIUS_API_KEY="your_nebius_api_key_here"
LANGCHAIN_API_KEY="your_langsmith_api_key_here"
TAVILY_API_KEY="your_tavily_api_key_here"
```

```
"""
黑板架构（Blackboard Architecture）- 基于共享知识库的多智能体协作系统

黑板架构是一种经典的智能体协作模式，其核心思想是：
- 多个专家智能体通过共享的"黑板"（共享状态）进行通信和协作
- 每个专家独立工作，将其分析结果发布到黑板上
- 其他专家可以读取黑板上的信息，进行增量分析
- 控制器（Controller）协调专家的工作顺序和终止条件
"""

import os
from typing import List, Annotated, TypedDict, Optional
from dotenv import load_dotenv

# ==================== LangChain 组件导入 ====================
# LangChain提供LLM抽象、工具集成和提示模板等核心功能
from langchain_nebius import ChatNebius          # Nebius云平台的大语言模型接
口
from langchain_tavily import TavilySearch        # Tavily搜索引擎工具，用于获
取实时网络信息
from langchain_core.messages import BaseMessage, SystemMessage,
HumanMessage  # 消息类型定义
from pydantic import BaseModel, Field            # Pydantic用于数据验证和结构
化输出定义
from langchain_core.prompts import ChatPromptTemplate  # 结构化提示模板

# ==================== LangGraph 组件导入 ====================
# LangGraph提供基于图的状态机框架，用于构建复杂的多智能体工作流
from langgraph.graph import StateGraph, END      # 状态图核心组件和终止节点
```

```python
# ==================== 可视化与调试工具 ====================
# Rich库提供增强的终端输出格式，便于调试和演示
from rich.console import Console                    # 富文本控制台输出
from rich.markdown import Markdown                  # Markdown格式渲染

# ==================== API密钥与环境配置 ====================
"""
黑板架构环境变量配置说明：
- NEBIUS_API_KEY: Nebius LLM服务的认证密钥
- LANGCHAIN_API_KEY: LangChain追踪服务的认证密钥
- TAVILY_API_KEY: Tavily搜索API的访问密钥
- LANGCHAIN_PROJECT: LangChain追踪项目标识符（设置为黑板架构专用项目）
黑板架构的追踪配置独立于之前的架构（ReAct、Multi-Agent、PEV），便于对不同架构的性能和表现进行对比分析。
"""

load_dotenv()  # 从.env文件加载环境变量

# 配置LangChain追踪系统，用于监控和调试黑板架构智能体行为
os.environ["LANGCHAIN_TRACING_V2"] = "true"       # 启用V2版本的追踪功能
os.environ["LANGCHAIN_PROJECT"] = "Agentic Architecture - Blackboard (Nebius)"  # 设置黑板架构专用追踪项目

# 验证关键环境变量是否存在，缺失时给出提示，这三个API密钥是系统正常运行的必要条件
for key in ["NEBIUS_API_KEY", "LANGCHAIN_API_KEY", "TAVILY_API_KEY"]:
    if not os.environ.get(key):
        print(f"{key} not found. Please create a .env file and set it.")

print("Environment variables loaded and tracing is set up.")
```

阶段 1：基线对比——顺序多智能体系统

为了理解黑板系统的灵活性，首先需要一个**能正常运行的顺序系统**。原始版本失败的原因是专业智能体没有利用前一步的输出结果。我们将通过**确保每个智能体从状态中获取必要的上下文**来修正这一问题。

步骤 1.1：构建顺序团队

定义专业智能体，使其**显式使用前驱智能体的输出结果**，然后以**固定的线性顺序**将它们连接起来。

```python
# ==================== 控制台与LLM初始化 ====================
# 初始化Rich控制台实例，用于增强终端输出格式和可视化效果
console = Console()

# 使用更具能力的模型处理复杂指令
# Mixtral 8x22B是混合专家模型(MoE)，参数量大，指令遵循能力强
# temperature=0确保输出确定性，便于调试和对比
llm = ChatNebius(model="mistralai/Mixtral-8x22B-Instruct-v0.1",
temperature=0)
```

```python
# 初始化Tavily搜索工具，限制最多返回2条结果以控制token消耗
search_tool = TavilySearch(max_results=2)

# ==================== 串行多智能体状态定义 ====================
# 定义串行流水线架构的状态数据结构
# 与之前的Multi-Agent架构类似，但执行顺序有差异：
# - 之前的Multi-Agent: 新闻 → 技术 → 财务 → 编写
# - 本架构的关键改进：技术分析和财务分析都基于新闻报告
class SequentialState(TypedDict):
    """
    串行多智能体系统状态数据结构
    关键设计决策：技术分析和财务分析都依赖新闻报告，形成信息传递链
    """
    user_request: str                    # 原始用户请求，保持不变
    news_report: Optional[str]           # 新闻分析报告（第一个执行）
    technical_report: Optional[str]      # 技术分析报告（依赖新闻）
    financial_report: Optional[str]      # 财务分析报告（依赖新闻）
    final_report: Optional[str]          # 最终综合报告

# ==================== 串行架构专家节点定义 ====================
# 核心改进：每个专家节点现在获取前序步骤的上下文，而非仅原始请求，体现了信息在流水线中的传递和积累

def news_analyst_node_seq(state: SequentialState):
    """
    新闻分析专家节点 - 串行流水线的第一阶段
    职责：搜索与用户请求相关的最新新闻；提供简洁的新闻摘要
    输出将作为技术分析和财务分析的上下文输入

    参数:state: 当前状态，包含用户请求
    返回:dict: 包含新闻报告的状态更新
    """
    console.print("--- (Sequential) CALLING NEWS ANALYST ---")

    # 构建提示词：明确角色定位和任务要求
    prompt = f"Your task is to act as an expert News Analyst. Find the latest major news about the topic in the user's request and provide a concise summary.\n\nUser Request: {state['user_request']}"

    # 绑定搜索工具，使智能体能够获取实时信息
    agent = llm.bind_tools([search_tool])
    result = agent.invoke(prompt)

    return {"news_report": result.content}

def technical_analyst_node_seq(state: SequentialState):
    """
    技术分析专家节点 - 串行流水线的第二阶段
    关键改进：该节点现在使用新闻报告作为上下文输入，体现了信息传递的价值：技术分析基于最新新闻进行
    职责：基于新闻报告，分析公司股票的技术面（价格趋势、技术指标等）

    参数:state: 当前状态，包含新闻报告
    返回:dict: 包含技术分析报告的状态更新
```

```
    """
    console.print("--- (Sequential) CALLING TECHNICAL ANALYST ---")

    # 关键改进：使用新闻报告作为分析上下文
    prompt = f"Your task is to act as an expert Technical Analyst. Based on
the following news report, conduct a technical analysis of the company's
stock.\n\nNews Report:\n{state['news_report']}"

    agent = llm.bind_tools([search_tool])
    result = agent.invoke(prompt)

    return {"technical_report": result.content}

def financial_analyst_node_seq(state: SequentialState):
    """
    财务分析专家节点 - 串行流水线的第二阶段（与技术分析并行）
    关键改进：该节点同样使用新闻报告作为上下文输入，体现了信息复用的价值：同一份新闻报告
支持多个分析维度
    职责：基于新闻报告，分析公司的财务绩效

    参数:state: 当前状态，包含新闻报告
    返回:dict: 包含财务分析报告的状态更新
    """
    console.print("--- (Sequential) CALLING FINANCIAL ANALYST ---")

    # 关键改进：使用新闻报告作为分析上下文
    prompt = f"Your task is to act as an expert Financial Analyst. Based on
the following news report, analyze the company's recent financial
performance.\n\nNews Report:\n{state['news_report']}"

    agent = llm.bind_tools([search_tool])
    result = agent.invoke(prompt)

    return {"financial_report": result.content}

def report_writer_node_seq(state: SequentialState):
    """
    报告编写节点 - 串行流水线的最终阶段
    职责：整合三个专家的分析报告，形成直接回答用户请求的连贯报告

    参数:state: 当前状态，包含所有专家报告
    返回:dict: 包含最终报告的状态更新
    """
    console.print("--- (Sequential) CALLING REPORT WRITER ---")

    # 构建整合提示词，将所有专家报告组合
    prompt = f"""You are an expert report writer. Your task is to
synthesize the information from the News, Technical, and Financial analysts
into a single, cohesive report that directly answers the user's original
request.

User Request: {state['user_request']}

Here are the reports to combine:
```

```
---
News Report: {state['news_report']}
---
Technical Report: {state['technical_report']}
---
Financial Report: {state['financial_report']}
"""
    report = llm.invoke(prompt).content
    return {"final_report": report}

# ==================== 串行流水线图构建 ====================
# 构建具有固定执行顺序的串行流水线图
# 与之前Multi-Agent架构的区别:
# - 之前: 新闻 → 技术 → 财务 → 编写 (技术、财务独立于新闻)
# - 现在: 新闻 → 技术、财务 (都依赖新闻) → 编写

seq_graph_builder = StateGraph(SequentialState)

# 添加四个核心节点
seq_graph_builder.add_node("news", news_analyst_node_seq)           # 新闻分析节点
seq_graph_builder.add_node("tech", technical_analyst_node_seq)      # 技术分析节点
seq_graph_builder.add_node("finance", financial_analyst_node_seq)   # 财务分析节点
seq_graph_builder.add_node("writer", report_writer_node_seq)        # 报告编写节点

# 定义刚性、硬编码的执行序列, 体现了串行流水线的确定性执行模式
seq_graph_builder.set_entry_point("news")        # 入口点: 新闻分析
seq_graph_builder.add_edge("news", "tech")       # 新闻 → 技术分析
seq_graph_builder.add_edge("tech", "finance")    # 技术分析 → 财务分析
seq_graph_builder.add_edge("finance", "writer") # 财务分析 → 报告编写
seq_graph_builder.add_edge("writer", END)        # 编写完成后终止

# ==================== 图编译与应用初始化 ====================
# 将构建的串行流水线图编译为可执行的应用实例
sequential_app = seq_graph_builder.compile()

# 输出编译成功状态
print("Corrected sequential multi-agent system compiled successfully.")
```

步骤 1.2: 在动态问题上测试顺序智能体

现在顺序智能体已能**正确传递上下文**, 它将生成一份更连贯的报告, 但其**执行过程仍然低效**, 且未能遵循条件逻辑。

```
# ==================== 动态查询测试用例配置 ====================
# 构建包含条件分支的动态查询, 用于测试串行流水线架构的灵活性
# 该查询的关键特征: 需要先获取新闻信息, 根据新闻情感极性决定后续分析路径, 体现了"数据驱动决策"的工作流模式
```

```
#
# 查询设计意图:
# - 测试串行架构是否能够处理条件分支逻辑
# - 评估智能体能否根据中间结果动态调整分析策略
# - 验证信息传递机制的有效性
dynamic_query = """Find the latest major news about Nvidia.
Based on the sentiment of that news, conduct either a technical analysis
(if the news is neutral or positive) or a financial analysis of their
recent performance (if the news is negative)."""

# ==================== 测试执行与可视化 ====================
# 输出测试开始标识，使用黄色区分不同类型的测试
console.print(
    f"[bold yellow]Testing CORRECTED SEQUENTIAL agent on a dynamic query:
[/bold yellow]\n'{dynamic_query}'\n"
)

# ==================== 串行流水线系统调用 ====================
# 执行串行多智能体图应用，处理包含条件分支的动态查询。
#
# 执行流程详解:
# 1. 阶段1为新闻分析，由NEWS ANALYST负责，任务是搜索Nvidia最新重大新闻，使用
TavilySearch工具，输出包含内容、来源和情感倾向的新闻报告。
# 2. 阶段2为技术分析，由TECHNICAL ANALYST负责，条件为新闻情感为中性或积极时执行，任务
是基于新闻进行技术分析，包括价格趋势分析、技术指标分析和支撑阻力位判断，输出技术分析报告或
空报告。
# 3. 阶段3为财务分析，由FINANCIAL ANALYST负责，条件为新闻情感为负面时执行，任务是基于
新闻进行财务分析，包括收入利润分析、研发支出分析和财务健康度评估，输出财务分析报告或空报
告。
# 4. 阶段4为报告编写，由REPORT WRITER负责，输入为新闻报告、技术分析报告和财务分析报
告，任务是整合有效分析结果，识别哪些报告有有效内容，根据新闻情感选择主要分析框架，生成连贯
的最终报告，输出动态适应的综合报告。
#
# ==================== 串行架构的局限性分析 ====================
# 1. 刚性执行路径。所有节点都会执行，无论是否需要，技术分析和财务分析都会运行，即使只有一
个有意义，这样浪费计算资源，可能引入无关信息。
# 2. 缺乏条件路由。图结构是固定的线性序列，无法根据新闻情感动态选择分析路径，报告编写者需
要自行过滤无关内容。
# 3. 依赖智能体自主判断。技术分析师需要判断新闻是否适合技术分析，财务分析师需要判断新闻是
否适合财务分析，如果智能体判断失误，可能产生不相关分析。
# 4. 信息冗余风险。两个分析节点可能都生成内容，报告编写者需要处理潜在的信息冲突，可能导致
报告内容矛盾或混乱。
#
# 这正是黑板架构要解决的问题：控制器可以根据黑板状态动态选择下一个执行节点，专家只在有意义
时才被调用，避免执行不必要的分析步骤。
final_seq_output = sequential_app.invoke({"user_request": dynamic_query})

# ==================== 结果输出与渲染 ====================
# 输出最终报告的分隔标识
console.print("\n--- [bold red]Final Report from Sequential Agent[/bold
red] ---")

# 使用Markdown渲染串行智能体生成的最终报告
# 期望的输出行为:
```

```
# - 如果新闻情感积极/中性：报告主要包含技术分析；如果新闻情感负面：报告主要包含财务分析
# - 可能同时包含两种分析（由于串行架构的限制），报告编写者会尝试整合所有可用信息
console.print(Markdown(final_seq_output['final_report']))
```

> **输出结果讨论：** 该智能体现在能够生成一份**完整、逻辑合理的报告**。然而，执行轨迹新闻 → 技术 → 财务揭示了其根本缺陷。它**同时执行**了技术分析和财务分析，完全忽略了用户的**条件性请求**（“要么…… 要么……”）。这是低效的，也体现了我们试图通过**黑板架构**解决的**僵化性问题**。

阶段 2：进阶方案——黑板系统

构建黑板系统。修正原始版本循环行为的关键在于为**控制器**设计一个**更加智能的提示词**，使其意识到自己作为**有状态规划器**的角色。

步骤 2.1：定义黑板与控制器

1. **黑板状态：** 定义一个 `BlackboardState` 作为共享内存的数据结构。
2. **专业智能体：** 定义各个**专业智能体节点**。它们与我们之前的智能体类似。
3. **控制器（修正版）：** 创建一个健壮的 `controller_node`，其提示词需要明确地对**已完成步骤和剩余目标进行推理**。这是最关键的变化。

```
# ==================== 黑板架构状态定义 ====================
# 定义黑板架构的核心状态数据结构,黑板(Blackboard)是专家之间通信和知识共享的中央存储区
class BlackboardState(TypedDict):
    """
    黑板架构状态数据结构
    """
    user_request: str                    # 原始用户请求
    blackboard: List[str]                # 中央黑板：存储所有专家的分析结果
    available_agents: List[str]          # 可用专家列表（如['News Analyst',
...])
    next_agent: Optional[str]            # 控制器选择的下一个专家

# ==================== 控制器决策模型 ====================
# 定义控制器的结构化输出格式，用于智能调度专家
class ControllerDecision(BaseModel):
    """
    控制器决策模型 - 实现动态专家调度
    控制器负责：分析当前黑板状态和用户请求；决定下一个最有价值的专家；判断何时任务完成可
以终止
    相比固定流水线这种设计实现了：自适应调度+按需调用+灵活终止
    """
    next_agent: str = Field(
        description="The name of the next agent to call. Must be one of
['News Analyst', 'Technical Analyst', 'Financial Analyst', 'Report Writer']
or 'FINISH'."
        # 可选值：四个专家之一 或 FINISH（任务完成）
    )
    reasoning: str = Field(
        description="A brief reason for choosing the next agent."
```

```
        # 决策理由，增强可解释性
    )

# ==================== 黑板专家工厂函数 ====================
# 创建可在黑板架构中工作的专家节点，关键特征：所有专家都能读取黑板上的已有信息
def create_blackboard_specialist(persona: str, agent_name: str):
    """
    黑板专家节点工厂函数
    创建的专家能力：读取黑板上的所有已有信息；理解原始用户请求；使用工具获取新信息；将分析结果发布回黑板
    与串行架构专家的关键区别：
    - 串行架构：专家只能看到前序步骤的输出
    - 黑板架构：专家可以看到所有已发布的信息

    参数:persona: 专家的角色描述，定义其专业领域
        agent_name: 专家标识符，用于签名报告
    返回:callable: 黑板专家节点函数
    """

    # 构建系统提示词
    # 关键指令：读取黑板内容作为上下文+使用工具获取所需信息+用专家名称签名报告
    system_prompt = f"""You are an expert specialist agent: a {persona}. Your task is to contribute to a larger goal by performing your specific function.
Read the initial User Request and the current Blackboard for context.
Use your tools to find the required information.
Finally, post your concise markdown report back to the blackboard. Your report should be signed with your name '{agent_name}'.
"""

    # 构建提示模板，包含用户请求和黑板内容
    prompt_template = ChatPromptTemplate.from_messages([
        ("system", system_prompt),
        ("human", "User Request: {user_request}\n\nBlackboard (previous reports):\n{blackboard_str}")
    ])

    # 创建处理链：提示模板 + 绑定工具的LLM
    agent = prompt_template | llm.bind_tools([search_tool])

    def specialist_node(state: BlackboardState):
        """
        黑板专家节点执行函数
        工作流程：将黑板内容格式化为字符串 -> 调用LLM处理用户请求和黑板上下文 -> 生成带签名的分析报告 -> 将报告追加到黑板

        参数:state: 当前黑板状态
        返回:dict: 更新后的黑板（追加新报告）
        """
        console.print(f"--- (Blackboard) AGENT '{agent_name}' is working... ---")

        # 将黑板内容格式化为字符串，用分隔符区分不同报告
        blackboard_str = "\n---\n".join(state["blackboard"])
```

```
        # 调用处理链
        result = agent.invoke({
            "user_request": state["user_request"],
            "blackboard_str": blackboard_str
        })

        # 生成带专家签名的报告
        report = f"**Report from {agent_name}:**\n{result.content}"

        # 将新报告追加到黑板
        return {"blackboard": state["blackboard"] + [report]}

    return specialist_node

# ==================== 黑板专家实例化 ====================
# 创建四个黑板专家节点
# 每个专家拥有不同的专业领域和工具访问权限

# 1. 新闻分析专家：负责信息搜集和市场情绪分析
news_analyst_bb = create_blackboard_specialist("News Analyst",  "News
Analyst")
# 2. 技术分析专家：负责价格趋势和技术指标分析
technical_analyst_bb = create_blackboard_specialist("Technical Analyst",
"Technical Analyst")
# 3. 财务分析专家：负责财务报表和基本面分析
financial_analyst_bb = create_blackboard_specialist("Financial Analyst",
"Financial Analyst")
# 4. 报告编写专家：负责综合黑板上的所有信息生成最终答案
report_writer_bb = create_blackboard_specialist("Report Writer who
synthesizes a final answer from the blackboard",  "Report Writer")

# ==================== 智能控制器节点 ====================
def controller_node(state: BlackboardState):
    """
    智能控制器节点 - 黑板架构的核心决策引擎
    控制器负责：分析当前黑板状态和用户请求；评估哪些信息已经收集，哪些信息仍然缺失；决定
下一个最有价值的专家；判断何时任务完成
    调度策略：优先收集基础信息（新闻）、根据已有信息决定后续分析、只有在信息充分时才调用报
告编写者

    参数:state: 当前黑板状态
    返回:dict: 更新next_agent字段，指示下一个要执行的专家
    """
    console.print("--- CONTROLLER: Analyzing blackboard... ---")

    # 使用结构化输出LLM做出调度决策
    controller_llm = llm.with_structured_output(ControllerDecision)

    # 格式化黑板内容
    blackboard_content = "\n\n".join(state['blackboard'])
    agent_list = state['available_agents']

    # 增强提示词：状态感知、目标导向
```

```
    # 关键改进：控制器现在能够理解任务的当前进度
    prompt = f"""You are the central controller of a multi-agent system.
Your job is to analyze the shared blackboard and the original user request
to decide which specialist agent should run next.

**Original User Request:**
{state['user_request']}

**Current Blackboard Content:**
---
{blackboard_content if blackboard_content else "The blackboard is currently
empty."}
---

**Available Specialist Agents:**
{', '.join(agent_list)}

**Your Task:**
1.  Read the user request and the current blackboard content carefully.
2.  Determine what the *next logical step* is to move closer to a complete
answer.
3.  Choose the single best agent to perform that step from the list of
available agents.
4.  If the user's request has been fully addressed and a final report has
been written, choose 'FINISH'. Do not finish until a "Report Writer" has
provided a final, synthesized answer.

Provide your decision in the required format.
"""

    # 调用控制器LLM做出决策
    decision_result = controller_llm.invoke(prompt)

    console.print(f"--- CONTROLLER: Decision is to call
'{decision_result.next_agent}'. Reason: {decision_result.reasoning} ---")

    # 更新状态中的next_agent字段，LangGraph将根据此字段路由
    return {"next_agent": decision_result.next_agent}

print("Blackboard components and corrected Controller node defined.")
```

步骤 2.2：构建黑板工作流图

将各个组件连接成一个**动态图**。**控制器充当中枢路由器的角色**。在任何专业智能体运行完成后，控制权始终返回给**控制器**，由其**决定下一步操作**。

```
# ==================== 黑板架构图构建 ====================
# 初始化黑板架构的状态图，使用BlackboardState作为状态容器
# 该图实现了一个动态调度的多智能体协作系统
bb_graph_builder = StateGraph(BlackboardState)
```

```python
# ==================== 添加所有节点 ====================
# 将控制器节点和四个专家节点添加到计算图中

# 添加控制器节点 - 系统的核心决策引擎，负责分析黑板状态并决定下一个要执行的专家
bb_graph_builder.add_node("Controller", controller_node)

# 添加四个黑板专家节点，每个专家都能读取黑板上的所有信息并发布自己的分析结果
bb_graph_builder.add_node("News Analyst", news_analyst_bb)           # 新闻
分析专家
bb_graph_builder.add_node("Technical Analyst", technical_analyst_bb) # 技术
分析专家
bb_graph_builder.add_node("Financial Analyst", financial_analyst_bb) # 财务
分析专家
bb_graph_builder.add_node("Report Writer", report_writer_bb)         # 报告
编写专家

# 设置入口点：从控制器开始
# 控制器首先分析黑板状态（初始为空），决定第一个执行的专家
bb_graph_builder.set_entry_point("Controller")

# ==================== 路由函数定义 ====================
# 定义基于控制器决策的动态路由逻辑
def route_to_agent(state: BlackboardState):
    """
    路由决策函数 - 根据控制器决策返回下一个节点名称
    从状态中读取next_agent字段，该字段由控制器节点设置。
    这是实现动态调度的关键：控制器决定调用哪个专家；路由函数将该决策映射到实际的节点名称

    参数:state: 当前黑板状态，包含next_agent字段
    返回值:str: 下一个要执行的节点名称（专家名或FINISH）
    """
    return state["next_agent"]

# ==================== 条件边配置 ====================
# 从控制器节点出发的条件边，根据控制器的决策进行路由
bb_graph_builder.add_conditional_edges(
    "Controller",                    # 源节点：控制器
    route_to_agent,                  # 路由函数：从状态中读取next_agent
    {
        # 路由映射：将控制器决策映射到实际节点名称
        "News Analyst": "News Analyst",           # 新闻分析专家
        "Technical Analyst": "Technical Analyst", # 技术分析专家
        "Financial Analyst": "Financial Analyst", # 财务分析专家
        "Report Writer": "Report Writer",         # 报告编写专家
        "FINISH": END                             # 任务完成，终止执行
    }
)

# ==================== 返回边配置 ====================
# 关键设计：任何专家执行完成后，控制权都返回给控制器
# 这形成了一个"控制器-专家-控制器"的循环，直到控制器决定FINISH

# 新闻分析专家完成后，返回控制器进行下一步决策
bb_graph_builder.add_edge("News Analyst", "Controller")
```

```
# 技术分析专家完成后，返回控制器进行下一步决策
bb_graph_builder.add_edge("Technical Analyst", "Controller")
# 财务分析专家完成后，返回控制器进行下一步决策
bb_graph_builder.add_edge("Financial Analyst", "Controller")
# 报告编写专家完成后，返回控制器进行下一步决策
# 控制器将判断是否还有更多工作要做，或者决定FINISH
bb_graph_builder.add_edge("Report Writer", "Controller")

# ==================== 图编译与应用初始化 ====================
# 将构建的黑板图结构编译为可执行的应用实例
blackboard_app = bb_graph_builder.compile()
# 输出编译成功状态
print("Blackboard system compiled successfully.")
```

阶段 3：直接对比

在新的黑板系统上运行**相同的动态任务**，并观察其智能的工作流程。

```
# ==================== 黑板架构测试执行 ====================
# 对黑板架构系统进行相同动态查询的测试
# 该测试旨在验证黑板架构相比串行流水线在处理动态条件分支时的优越性
console.print(
    f"[bold green]Testing BLACKBOARD system on the same dynamic query:
[/bold green]\n'{dynamic_query}'\n"
)

# ==================== 初始化黑板状态 ====================
# 配置黑板系统的初始状态
agent_list = ["News Analyst", "Technical Analyst", "Financial Analyst",
"Report Writer"]

# 构建初始输入状态
initial_bb_input = {
    "user_request": dynamic_query,        # 原始用户请求（包含条件分支逻辑）
    "blackboard": [],                     # 黑板初始为空，尚未有任何专家发布报告
    "available_agents": agent_list        # 可用专家列表，供控制器调度选择
}

# ==================== 流式执行与观察 ====================
# 使用stream方法逐步观察黑板系统的执行过程
# recursion_limit=10：限制最大递归深度，防止控制器陷入无限循环（如专家之间互相依赖导致
无法终止），这是保护系统稳定性的重要安全机制
final_bb_output = None
for chunk in blackboard_app.stream(initial_bb_input, {"recursion_limit":
10}):
    final_bb_output = chunk

    # ==================== 黑板状态可视化 ====================
    # 在每个步骤后输出当前黑板状态，便于理解系统执行过程
    console.print("\n--- [bold purple]Current Blackboard State[/bold
```

```
purple] ---")

    # 遍历并格式化输出黑板上的所有报告，每个报告都带有专家签名，便于追踪信息来源
    for i, report in enumerate(final_bb_output.get('blackboard', [])):
        console.print(f"--- Report {i+1} ---")
        console.print(Markdown(report))
    console.print("\n")

# ==================== 执行流程分析 ====================
# 黑板架构对动态查询的预期执行流程如下。
# 【迭代1 - 控制器决策】
# CONTROLLER 分析黑板（空），用户请求需要先获取新闻信息，新闻情感将决定后续分析路径，决策为调用 News Analyst。
# News Analyst 执行，搜索 Nvidia 最新重大新闻，发布带签名的新闻报告到黑板，黑板状态变为包含新闻报告。
# 【迭代2 - 控制器决策】
# CONTROLLER 分析黑板（包含新闻报告），读取新闻内容，分析情感倾向。场景A为新闻积极或中性时调用 Technical Analyst，场景B为新闻负面时调用 Financial Analyst，场景C为新闻复杂时可能调用多个专家。
# 相关专家执行（根据新闻情感选择），专家读取黑板上的新闻报告作为上下文，执行专业分析，发布带签名的分析报告到黑板。
# 【迭代3 - 控制器决策】
# CONTROLLER 分析黑板（包含新闻和分析报告），评估是否已有足够信息回答用户请求，判断是否需要综合报告，决策为调用 Report Writer（如果需要综合）或 FINISH。
# Report Writer 执行，综合黑板上的所有报告，生成符合用户请求的最终答案，发布带签名的综合报告到黑板。
# 【迭代4 - 控制器决策】
# CONTROLLER 分析黑板（包含综合报告），用户请求已被完整回答，决策为 FINISH，执行终止。

# ==================== 最终结果输出 ====================
# 输出最终报告的分隔标识
console.print("\n--- [bold green]Final Report from Blackboard System[/bold green] ---")

# 提取最终报告内容
# 黑板上的最后一条报告通常是 Report Writer 发布的综合报告
final_report_content = final_bb_output['blackboard'][-1]

# 使用Markdown渲染最终报告
console.print(Markdown(final_report_content))
```

> **输出结果讨论：** 成功！`GraphRecursionError` 错误已消失。执行轨迹揭示了一个更加智能的处理过程：
>
> 1. **控制器启动：** 控制器启动后，看到黑板为空，正确决定首先调用**新闻分析师**。
> 2. **新闻分析师运行：** 新闻分析师找到最新新闻，并将**其报告发布到黑板上**。
> 3. **控制器重新评估：** **控制权返回给控制器**。它读取新闻分析师的报告，理解市场情绪，并遵循用户设定的逻辑。它**智能地决定调用接下来合适的分析师（技术分析师或财务分析师）**，完全跳过了另一个。
> 4. **专业智能体运行：** 被选中的分析师执行其任务，并将其报告添加到黑板上。
> 5. **控制器完成：** 控制器看到所有必要的分析均已完成，便调用**报告撰写者**来整合最终答案。

> 6. **最终调用：** 在报告撰写者发布最终报告后，控制器看到这一结果并决定**结束**。
>
> 这种**动态的**、**机会主义的**工作流程是正常运作的黑板系统的标志。它完美地遵循了用户复杂的条件逻辑，既节省了时间，也节约了资源。

阶段 4：定量评估

为了将对比结果形式化，采用“**大语言模型即评审**”的方式，对两种系统在指令遵循度和过程效率方面进行评分。

```
# ==================== 过程逻辑评估模型定义 ====================
# 定义智能体过程逻辑评估模型，专门评估智能体如何执行任务
# 与之前的结果评估不同，该评估关注执行过程而非最终输出质量
class ProcessLogicEvaluation(BaseModel):
    """
    智能体过程逻辑评估模型
    该评估框架从两个维度评估智能体的执行过程：
    1. 指令遵循度：评估智能体是否理解并执行了用户的条件逻辑
    2. 过程效率：评估智能体是否避免了不必要的计算和资源浪费

    属性:instruction_following_score: 指令遵循度评分（1-10）
            评估智能体是否正确处理条件分支逻辑
        process_efficiency_score: 过程效率评分（1-10）
            评估智能体是否采取了最直接的路径，避免执行用户明确要求跳过的专家
        justification: 评分理由
            引用具体的执行步骤，提供可解释的评估依据
    """
    instruction_following_score: int = Field(
        description="Score 1-10 on how well the agent followed the user's
specific conditional instructions (e.g., the 'either/or' logic)."
    )
    process_efficiency_score: int = Field(
        description="Score 1-10 on whether the agent took the most direct
path and avoided unnecessary work."
    )
    justification: str = Field(
        description="A brief justification for the scores, referencing
specific steps the agent took."
    )

# ==================== 评估LLM配置 ====================
# 使用更强大的模型进行过程评估，结构化输出确保评估结果格式一致
judge_llm = ChatNebius(
    model="mistralai/Mixtral-8x22B-Instruct-v0.1",
    temperature=0
).with_structured_output(ProcessLogicEvaluation)

# ==================== 智能体过程逻辑评估函数 ====================
def evaluate_agent_logic(query: str, final_state: dict) ->
ProcessLogicEvaluation:
    """
    智能体过程逻辑评估函数 - 评估智能体的执行过程质量
```

```
    该函数专门评估智能体如何处理包含条件逻辑的任务，而非评估最终答案的内容质量。这对于比
较不同架构的执行效率和行为正确性至关重要。

    参数:query: 原始用户请求，包含条件逻辑（如"either...or..."）
        final_state: 智能体的最终状态
            - 黑板架构: 包含blackboard字段
            - 串行架构: 包含各专家报告字段
    返回:ProcessLogicEvaluation: 包含指令遵循度和过程效率的评估结果
    """

    # 根据智能体类型重建执行轨迹，轨迹格式必须清晰展示执行了哪些步骤
    trace = ""
    agent_type = "Unknown"

    # 检测智能体类型：黑板架构或串行架构
    if 'blackboard' in final_state:  # 黑板架构智能体
        agent_type = "Blackboard"
        # 将黑板上的所有报告用分隔符连接
        # 每个报告都带有专家签名，便于识别调用了哪些专家
        trace = "\n---\n".join(final_state['blackboard'])
    else:  # 串行架构智能体
        agent_type = "Sequential"
        # 构建串行架构的执行轨迹
        # 格式：序号 + 报告类型 + 报告内容
        trace = f"""1. News Report Generated:
{final_state.get('news_report')}
---
2. Technical Report Generated: {final_state.get('technical_report')}
---
3. Financial Report Generated: {final_state.get('financial_report')}"""

    # 构建评估提示词
    # 明确要求评估两个维度：是否遵守了条件逻辑（either/or）；是否避免了不必要的工作
    prompt = f"""You are an expert judge of AI agent processes. Your task
is to evaluate an agent's performance based on its generated content trace.

**User's Original Task:**
"{query}"

**Agent's Type:** {agent_type}
**Agent's Generated Content Trace:**
\`\`\`
{trace}
\`\`\`

**Evaluation Criteria:**
1.  **Instruction Following:** Did the agent respect the conditional logic
in the user's task? (e.g., "either a technical analysis... or a financial
analysis"). A high score means it followed the logic perfectly. A low score
means it ignored it.
2.  **Process Efficiency:** Did the agent avoid doing unnecessary work? A
high score means it only ran the required specialists. A low score means it
ran specialists that the user's logic explicitly said to skip.
```

```
Based on the trace, provide your evaluation.
"""

    # 调用评估LLM
    return judge_llm.invoke(prompt)

# ==================== 串行架构过程评估 ====================
# 对串行流水线智能体进行过程逻辑评估
# 预期结果：串行架构会执行所有专家（新闻、技术、财务、编写），这违反了"either/or"的条件逻辑 -> 过程效率评分应该较低
console.print("--- [bold]Evaluating Sequential Agent's Process[/bold] ---")
seq_agent_evaluation = evaluate_agent_logic(dynamic_query, final_seq_output)
console.print(seq_agent_evaluation.dict())

# ==================== 黑板架构过程评估 ====================
# 对黑板智能体进行过程逻辑评估
# 预期结果：黑板架构会根据新闻情感动态选择专家，只执行必要的专家（新闻 + 一个分析专家 + 编写） -> 指令遵循度和过程效率评分应该较高
console.print("\n--- [bold]Evaluating Blackboard System's Process[/bold] ---")
bb_agent_evaluation = evaluate_agent_logic(dynamic_query, final_bb_output)
console.print(bb_agent_evaluation.dict())
```

> **评估输出结果讨论：** 评审模型的评分给出了明确的定量结论：
>
> - **顺序智能体**将获得**非常低的指令遵循度得分**（例如 2/10），因为它明显忽略了“二者择一”的条件。其**过程效率得分也会很低**（例如 3/10），因为它执行了明确不需要的完整分析。
> - **黑板系统**在两项指标上都将获得**接近满分的评分**（例如 10/10）。评审模型会认可，**控制器的动态决策使系统能够精确遵循用户指令，并且通过仅激活必要的专业智能体**，实现了最大程度的效率。
>
> 这一评估提供了确凿的证据，表明对于**执行路径依赖于中间结果的复杂、涌现性问题**，**黑板架构的灵活性**远优于僵化的、预定义的工作流。

结论

在本节笔记本中，我们实现了一个**黑板系统**，展示了其相较于**顺序多智能体**架构的显著优势。通过引入**共享内存（黑板）和一个智能的、具备状态感知能力的控制器**，我们构建了一个不仅具备**协作能力**，还具有**适应性和机会主义**特性的系统。

直接对比表明，对于带有**条件逻辑**的任务，黑板系统能够在正确的时间选择正确的专家，从而实现**更高效、逻辑更严谨**的执行过程。尽管它需要一个更精妙的**控制器**，但这种架构是应对结构不良、线性工作流无法有效解决的真实世界问题的强大工具。